

BIDSification of OPM MEG data

Pygers-MEG 2026

Raw data files: Cerca

From cAcq

File	What it is	What's inside
_meg_001.cMEG	The data. Every channel, sample by sample.	Continuous raw voltages for all 222 channels at 1200Hz. The _001 is a part number: a long recording can be split into _002, _003, and so on.
_meg.json	BIDS-style recording sidecar.	Sampling rate, channel counts (198 MEG, 8 trigger, 16 misc), duration. Empty descriptive fields. Invalid BIDS values: PowerLineFrequency=0, RecordingType="Noise"
_channels.tsv	Channel list, one row per channel, in BIDS-style columns.	222 channels: 198 sensor channels (66x3), 8 triggers, 16 misc. Units are volts. Status="undetermined". One extra non-BIDS column gives each sensor's gain (2.7 V/nT).
_HelmConfig.tsv	Helmet map: where each sensor sits and which way each axis points.	One row per helmet slot and axis. 64 slots named like EEG positions (F1, C3, Fpz, etc). Each row has: position (Px Py Pz, meters, helmet frame); the direction the axis measures (Ox Oy Oz, unit vector); 2D layout coordinates for topographic plots (Layx Layy); and which sensor is plugged into that slot (e.g. F1 = 3J). The last line names the helmet, Adult_L_purple_lite.
_SensorTransform.tsv	4x4 transform (rotation and translation) for the sensor positions.	Just an identity matrix (no change). Used when converting empty room recording. We will use transforms created by cREG when converting subject data.
_SessionInfo.txt	Session notes in plain text. Entered by researcher in cAcq.	Start time (in UTC), operator, Patient ID, ethics code, experiment type ("Noise", "Measurement", "Other"), whether the room was degaussed, helmet name, sensor gain, and free comments.

Raw data files: Cerca

From cReg

File	What it is	What's inside
_dev2head_xfm.tsv	Sensor-to-head scan transform: where the helmet (and every sensor) sat relative to the subject's head scan.	4x4 rigid-body matrix in meters. This will be used when converting cMEG to FIF.
_digitisation_from_mesh_3_xfmd.xyz	Head shape plus the 3 fiducials, as a list of 3D points.	Two header lines, and then ~50K points (x y z, meters, in head scan's coordinate system. The last 3 are NAS, LPA, RPA– the fiducials you mark in cReg. This is also passed on during conversion to FIF. The 3 fiducials define MNE's head coordinate system. Later these points are what you use to coregister the MRI.
_digitization_from_mesh_3_xfmd.ply	The head-only 3D scan itself as a surface mesh.	The above .xyz points come from this mesh ("mesh 3"). This is identifiable – you see the face. Keep out of anything shared. Will not be used. Store in sourcedata/ if at all.

- Empty room recording does not use these files during conversion. Just uses the identity matrix output by cAcq.
- “dev2head” is a bit misleading because the file maps sensors to the 3D scan's coordinate system, and not to MNE's head coordinate system. The true device-to-head transform is created after conversion to FIF when the transform is combined with the fiducials.
- The last step of cReg is to downsample the head shape points. We have been choosing 50K, but SQUID MEG pen digitizers have a few hundred points. It may be worth downsampling more because 50K can make MNE's coregistration and plotting slow.

cMEG2FIF

URL: <https://github.com/mrribbits/cMEG2FIF-Scully>

Python script to convert cMEG recordings to FIF.
Provided by Cerca, but no longer maintained by them.
We've updated it with a few useful options.

Installation options:

- A) pip install will install straight into your conda environment, install dependencies, and provide a command (cmeg2fif)
- B) Alternatively, download script directly, install dependencies manually, and run it with python.

Input files:

_meg_001.cMEG
_meg.json
_channels.tsv
_HelmConfig.tsv
_SessionInfo.txt

Additional inputs for subject recordings:

_dev2head_xfm.tsv
_digitization_from_mesh_3_xfmd.xyz

cMEG2FIF

Newly added features:

- Specify a peripherals tsv file to define trigger and BNC channels. They are all kept as individual channels.
- Generates a combined trigger channel and a combined button channel (STI101, STI_BTN).
- Writes the line frequency (60Hz default) into the FIF (raw.info['line_freq']) because Cerca writes 0.
- Writes the recording date/time (raw.info['meas_date']), pulled from SessionInfo.txt because Cerca doesn't.
- Optionally down samples head shape points.
- Conversion details recorded to a log.
- Never overwrites without passing --force.

It will write out the correct channel types:

mag : every on-head sensor axis, with position and orientation.

ref_mag : channels listed in channels.tsv with no helmet slot in _HelmConfig.tsv.

stim : the 8 Vpixx trigger lines, the 8 button channels, and the two combined channels (STI101, STI-BTN).

misc : additional BNC lines (e.g. eye tracker)

Example commands:

:: Empty-room recording (no coregistration)

```
python cMEG2fif.py 20260924_103257_meg_001.cMEG \  
    --peripherals cMEG_peripherals.tsv \  
    --no-plot \  
    --out 20260924_103257_meg_001.fif
```

:: Subject recording with head coregistration

```
python cMEG2fif.py 20260924_113141_meg_001.cMEG \  
    --xfm subject003_headHelmet_dev2head_xfm.tsv \  
    --dig subject003_headHelmet_digitisation_from_mesh_3_xfmd.xyz \  
    --peripherals cMEG_peripherals.tsv \  
    --out 20260924_113141_meg_001.fif
```

Once converted to FIF, is that it for BIDS?

No. Many items need to be generated.

MNE-BIDS

<https://mne.tools/mne-bids/stable/index.html>

About

MNE-BIDS is a Python package that allows you to read and write BIDS-compatible datasets with the help of MNE-Python.

NOTE: This is different from MNE-BIDS-Pipeline (<https://mne.tools/mne-bids-pipeline/stable/>)
Which is a complete analysis pipeline you configure and run on data **already** in BIDS.

Latest release: v0.20.0 (09/28/26)

MNE-BIDS

Recording-level

Item	BIDS requirement	MNE-BIDS does it?	Details
_meg.fif	Required	✓	
Folder layout	Required	✓	
Linking recording to empty room	Required	⦿	You must specify.
_meg.json	Required	⦿	Correct: SamplingFrequency (req), PowerLineFrequency (req), RecordingDuration (rec), RecordingType (rec), channel counts (rec), DigitizedLandmarks/HeadPoints (req), AssociatedEmptyRoom (rec). Wrong: Manufacturer (rec), DewarPosition (req), SoftwareFilters (req), HardwareFilters (rec) Missing (all rec): TaskDescription, Instructions, Model name, Institution fields, DeviceSerialNumber, SoftwareVersions
_channels.tsv	Recommended	⦿	Required and filled correctly: name, type, units. Optional and filled correctly: sampling rate, bad channels. Optional and filled incorrectly: Filter cutoffs are 0/600Hz-- set by MNE during FIF conversion b/c no values are provided in the Cerca channels.tsv.
_channels.json	Optional	✗	Never written.
_events.tsv + .json	Optional (but analysis needs it)	⦿	You must pass in events, decode trigger lines and buttons, etc.
_coordsystem.json	Optional (but coregistration needs it)	✓	
_scans.tsv	Optional	⦿	Only writes rows for the MEG files, but doesn't include the MRI scans.

MNE-BIDS

Dataset-level

Item	BIDS requirement	MNE-BIDS does it?	Details
Dataset_description.json	Required	🕒	Writes empty/generic fields. you have to fill.
README	Recommended	🕒	Writes a generic file, you have to fill.
Participants.tsv + json	Optional	🕒	Writes 5 default columns all with “n/a” since FIF has no subject info. You must fill yourself.
CHANGES	Optional	✗	
LICENSE	Optional	✗	
CITATION.cff	Optional	✗	
task-<label>_meg.json (top-level)	Optional	✗	

FLUX-pipeline

<https://github.com/FLUX-pipeline/Cerca>

<https://www.neurosc.com/flux>

About

The FLUX pipeline aims to make the analyses steps and setting explicit for standard analysis done in cognitive neuroscience. It focuses on quantifying and source localization of oscillatory brain activity, but it can also be used for event-related fields and multivariate pattern analysis.

Specifically, the pipeline including documented code is defined for MNE Python (a Python toolbox) and FieldTrip (a Matlab toolbox), and a data set on visuospatial attention is used to illustrate the steps.

The scripts are provided as notebooks implemented in Jupyter Notebook and MATLAB Live Editor providing explanations, justifications and graphical outputs for the essential steps.

Furthermore, we also provide suggestions for text and parameter settings to be used in registrations and publications to improve replicability and facilitate pre-registrations.

Latest release: v1.0 (08/30/2025)

mne-opm

Harrison Ritz

<https://github.com/harrisonritz/mne-opm>

About

OPM-MEG preprocessing pipeline based on MNE-Python, mne-bids-pipeline, and OSL-ephys. This repository contains utility scripts, custom pre-processing glue code, and run scripts to convert data to BIDS, run preprocessing, inspect and curate bad channels/epochs, run coregistration and FreeSurfer, and prepare sensor/source outputs.

Two scripts cover our BIDS steps: `run_bids.py` and `run_nifti.sh`.

The repo uses his own forks of mne-python, mne-bids, and mne-bids-pipeline.

Latest release: Dev

FLUX vs. mne-opm

Recording-level

Item	FLUX	mne-opm
Linking recording to empty room	✗ No empty room at all	✓ Fills it.
_meg.json: Wrong: Manufacturer (rec), DewarPosition (req), SoftwareFilters (req), HardwareFilters (rec)	✗ Not fixed.	✗ Not fixed
_meg.json: Missing (all rec): TaskDescription, Instructions, Model name, Institution fields, DeviceSerialNumber, SoftwareVersions	✗ Not fixed.	✗ Not fixed.
_channels.tsv: bad channels	✗ Bad sensors are marked in later notebooks, in memory only, never written back to BIDS	⦿ Bad channels from a config list are marked bad. No reason given (status_description stays n/a).
_channels.tsv: filter cutoffs	✗ Shows 0.1/250 Hz, which is FLUX's own filter, not the hardware bandwidth	✗ Same as MNE-BIDS (FIF defaults).
_channels.json	✗	✗
_events.tsv + .json	⦿ Combines Trigger 5–11 into codes, overwrites Trigger 1 with them, finds events and names them with an event_dict. Deletes all annotations. No button or log columns; the .json only has the default columns.	⦿ Furthest one. Decodes Trigger 1–8 into codes and named annotations, renames the button annotations (response_desc), keeps Cerca's other annotations, and re-checks trial counts after writing. No PsychoPy columns.
_scans.tsv	✗ (DICOMs are converted to NIfTI with dcm2niix, but never put into BIDS)	✗ It writes the MRI with write_anat, which adds no row to scans.tsv

FLUX vs. mne-opm

Dataset-level

Item	FLUX	mne-opm
Dataset_description.json	✗ Imports make_dataset_description but never calls it	✗ The converter doesn't write it. Only the generator for its synthetic example dataset writes a proper one, for that dataset alone.
README	✗	✗ Same: a hand-written README exists only for the synthetic dataset
Participants.tsv + json	✗	✗ Writes a subject ID into the FIF, so the row has participant_id with demographics still n/a
CHANGES	✗	✗
LICENSE	✗	✗
CITATION.cff	✗	✗
task-<label>_meg.json (top-level)	✗	✗

FLUX vs. mne-opm

Item	FLUX	mne-opm
Anonymization	✓	✗
Keeps MEG data untouched?	✗ filters 0.1-250Hz	✗ joins task runs, optional crop of starts
Combines triggers into codes?	● triggers 5-11 summed and written over trigger 1	● triggers 1-8 summed to generate event annotations and then all removed, leaving just the annotations.
Button presses	✗	● Set in subject config. No combined channel created.
Behav log merged into events.tsv?	✗	✗
Eye tracking	✗	● Merged/resampled into extra channels of FIF, not BIDS files, syncs using Eyelink Messages and Events, fits offset and drift, realigns
MRI	● dcm2nixx, no BIDS naming	● json sidecar not carried over

Cerca's cFIF program

1. Converts cMEG to FIF
2. Optionally launches cREG if you need to generate coregistration files.
3. Additional option to Save BIDS.

Uses an old version of MNE-BIDS from ~2022.

- Outdated BIDS spec.
- Same auto-generated template files.
- Doesn't correct line_freq.
- Doesn't correct manufacturer, etc.
- channels.tsv has no REF rows.
- Doesn't create combined trigger/button channels.
- Etc...

The screenshot displays the cFIF program interface, which is a window titled 'cFIF'. It features a tabbed interface with 'FIF Converter', 'Trigger Info', and 'Settings' tabs. The 'FIF Converter' tab is active, showing fields for 'MEG File', 'Helmet Config File', 'Digitisation File', 'Dev2head Transform File', and 'BIDS Path'. Each field has a 'Choose File' button. There are also 'Keep?' checkboxes and 'Confirm' buttons for the helmet config and digitisation files. A 'Run Coreg' button is present. Below these fields is a 'Trigger Info' section with a 'Confirm and Save' button and a refresh icon. It contains a grid of input fields for triggers (Trigger 1 to Trigger 8) and BNCs (BNC 1 Z to BNC 16 Z). To the right of the trigger info is a 'BIDS Info' section with input fields for 'Sub', 'Ses', 'Task', and 'Run'. At the bottom, there is a 'Save BIDS' button and a large text area showing the progress of the conversion process, including file copying, splitting, and writing. The status bar at the bottom indicates 'cFIF | v2.0.0'.

MEG File: C:/Users/mpinsk/Desktop/20260929_125813_cMEG_Data/20260929_125813_meg_001.cMEG Choose File

Helmet Config File: C:/Users/mpinsk/Desktop/20260929_125813_cMEG_Data/20260929_125813_HelmConfig.tsv Keep? Confirm

Digitisation File: C:/Users/mpinsk/Desktop/20260929_125813_cMEG_Data/20260929_125813_digitisation.xyz Keep? Confirm

Dev2head Transform File: C:/Users/mpinsk/Desktop/20260929_125813_cMEG_Data/20260929_125813_SensorTransform.tsv Run Coreg

Trigger Info: Confirm and Save ↺

Trigger 1: Trigger 1	Trigger 2: Trigger 2	Trigger 3: Trigger 3	Trigger 4: Trigger 4
Trigger 5: Trigger 5	Trigger 6: Trigger 6	Trigger 7: Trigger 7	Trigger 8: Trigger 8
BNC 1 Z: BNC 1 Z	BNC 2 Z: BNC 2 Z	BNC 3 Z: BNC 3 Z	BNC 4 Z: BNC 4 Z
BNC 5 Z: BNC 5 Z	BNC 6 Z: BNC 6 Z	BNC 7 Z: BNC 7 Z	BNC 8 Z: BNC 8 Z
BNC 9 Z: BNC 9 Z	BNC 10 Z: BNC 10 Z	BNC 11 Z: BNC 11 Z	BNC 12 Z: BNC 12 Z
BNC 13 Z: BNC 13 Z	BNC 14 Z: BNC 14 Z	BNC 15 Z: BNC 15 Z	BNC 16 Z: BNC 16 Z

BIDS Info:

Sub: 001

Ses: 001

Task: oddball

Run: 001

BIDS Path: C:/Users/mpinsk/Desktop/bids-test Save BIDS

Copying data files to sub-001_ses-001_task-oddball_run-001_meg.fif
Reserving possible split file sub-001_ses-001_task-oddball_run-001_split-01_meg.fif
Writing C:/Users/mpinsk/Desktop/bids-test/sub-001/ses-001/meg/sub-001_ses-001_task-oddball_run-001_meg.fif
Closing C:/Users/mpinsk/Desktop/bids-test/sub-001/ses-001/meg/sub-001_ses-001_task-oddball_run-001_meg.fif
[done]
Writing 'C:/Users/mpinsk/Desktop/bids-test/sub-001/ses-001/sub-001_ses-001_scans.tsv'
Wrote C:/Users/mpinsk/Desktop/bids-test/sub-001/ses-001/sub-001_ses-001_scans.tsv entry with meg/sub-001_ses-001_task-oddball_run-001_meg.fif.
No event file found....Proceeding anyway
Done!

Open Manual cFIF | v2.0.0

Explore sample dataset

<https://github.com/mrribbits/pygers-meg>

1. Download notebook.
2. Set up a conda environment on the cluster (or your laptop).
3. Copy sample dataset to your jukebox folder.
4. Follow notebook